

Simulation Repetition Module (SRPM)

OOF™ Origin Open Foundation™

Independent Methodological Authority

Architecture Ecosystem: Structured Reality Standards™

Architecture Family: SIMULOS® — Simulation Governance Architecture

Parent Standard: Simulation Execution Standard (SES)

Operational Layer: Simulation Execution Governance Layer

Category: Governance & Enforcement

Subcategory: Simulation Governance

Type: Parent Standard Module

Governed Space: Simulation Repetition

Version: 1.0

Status: Canonical · Open Module

Effective Date: 19 August 2026

Compatibility: OOF® Methodology OS™ · GOA™ · ORA™ · VALIDOS® · INTEGROS® · ADIT® · AGA™ · AIG® · ASGA™ · RIS™

AI-Readable: Yes

Authority: OOF®

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Governed Space

Simulation Repetition

Canonical Definition

Simulation Repetition Module (SRPM) defines the governance framework through which Simulation Executions are deliberately repeated under controlled, traceable, and sufficiently comparable conditions so that repeated runs can reveal stochastic variation, consistency, instability, sensitivity, rare behavior, execution divergence, and other materially relevant differences without allowing repetition volume to be mistaken for independent evidence or methodological strength.

SRPM governs:

what must remain constant, what may deliberately vary, why repetition is required, how repeated executions are related, and under which conditions their results may legitimately be compared or aggregated.

Its purpose is to transform repeated simulation from:

“run it again”

into:

a governed repetition process.

Operational Role

SRPM serves as the fourth internal governance space of the Simulation Execution Standard (SES).

The preceding modules establish:

SIM — What exact simulation instance was instantiated?

SECM — How was active execution controlled?

SOM — What materially relevant behavior and events were observed during execution?

SRPM asks:

When the simulation is repeated, which execution conditions must remain fixed, which may change, and how can the resulting runs remain meaningfully distinguishable and comparable?

The governed progression is:

Completed / Repeatable Simulation Execution

↓

Repetition Requirement

↓

Reference Execution

↓

Controlled Constant Conditions

↓

Permitted Variation

↓

Repetition Identity

↓

Repeated Simulation Executions



Repetition Comparability



Repetition Variance & Divergence



Repetition Sufficiency



Governed Simulation Repetition Set



Execution Documentation

Module Operational Space

SRPM governs: repetition requirements, repetition purpose, repetition identity, reference executions, repetition sets, repeat-run configuration, controlled constants, permitted variation, intentional variation, accidental variation, repeated execution binding, repeatability, reproducibility conditions, stochastic repetition, deterministic repetition, randomization, random seed governance, seed variation, initialization variation, execution-order variation, repetition sequencing, repetition independence, repetition dependence, repeated-run comparability, repeated-run equivalence, repetition divergence, repetition variance, rare-event emergence, convergence signals, repetition saturation, repetition sufficiency, repetition gaps, repetition failure, repetition drift, repetition versioning, repetition lineage, and repetition traceability.

Module Function

SRPM exists to prevent: repeated runs being treated as independent when they are not, identical runs being counted as broad simulation evidence, hidden parameter changes across repetitions, changed model or environment versions being treated as repetition of the same simulation, random seed changes disappearing, uncontrolled execution drift being mistaken for stochastic behavior, deterministic divergence being ignored, stochastic consistency being overstated, rare events being dismissed as anomalies without governance, successful repetitions being retained while failed repetitions disappear, arbitrary repetition counts being represented as methodological sufficiency, and large Monte Carlo campaigns being treated as meaningful merely because the run count is large.

Simulation Repetition

A Simulation Repetition is a subsequent governed Simulation Execution intentionally related to one or more prior executions for the purpose of examining behavior under materially equivalent or deliberately modified execution conditions.

Conceptually:

Reference Execution → Governed Repetition Conditions → Repeated Execution
Repetition Requirement

A Repetition Requirement establishes why simulation repetition is necessary.

Possible purposes include: examining stochastic variability, assessing consistency, determining execution stability, exploring uncertainty, evaluating sensitivity, identifying rare behavior, comparing alternative seeds, confirming unexpected observations, examining nondeterminism, evaluating distributional behavior, or supporting a broader simulation campaign.

No Repetition-for-Volume Principle

SIMULOS® establishes:

A simulation should not be repeated merely to inflate execution count where additional repetitions do not materially improve the governed simulation inquiry.

Reference Execution

A Reference Execution provides a defined execution basis against which subsequent repetitions may be compared.

It should remain traceable to: Execution ID, Simulation Object, model version, environment version, scenario, Input Set, parameter configuration, randomization state, execution controls, and relevant runtime conditions.

Reference Execution ≠ Correct Execution

A Reference Execution establishes comparison context.

It does not establish that the reference behavior was valid, expected, correct, or desirable.

Controlled Constants

A repetition process should identify which materially relevant execution conditions are intended to remain constant.

These may include: Simulation Object version, model version, environment version, Scenario ID, Input Set, parameter configuration, hardware state, software state, execution-control rules, initialization conditions, external dependencies, or other materially relevant conditions.

Controlled Constant Principle

A condition cannot support repetition comparability if the repetition process does not know whether that condition remained unchanged.

Permitted Variation

Some repetition dimensions may deliberately change.

Possible examples include: random seed, stochastic sampling, initial state, timing, actor behavior, uncertain parameter values, external-event ordering, or another explicitly governed variable.

Permitted variation should remain distinguishable from uncontrolled deviation.

Intentional vs Accidental Variation

SRPM establishes:

Intentional Variation → deliberately introduced to examine a defined simulation question.

Accidental Variation → arises without governed design or adequate control.

These two conditions must not be conflated.

Repetition Identity

Every repeated execution should retain its own execution identity while remaining connected to its repetition family.

Conceptually:

Repetition Set RS-001

may include:

Execution E-001

Execution E-002

Execution E-003

and:

Execution E-004

Each run remains independently traceable.

Repetition Set

A Simulation Repetition Set is a governed collection of related Simulation Executions performed under explicitly defined repetition conditions.

The set may identify: Reference Execution, repetition purpose, controlled constants, permitted variation, number of executions, randomization basis,

comparability conditions, and resulting execution statuses.

Repetition Set $\neq$ Scenario Set

A Scenario Set defines which conditions should be simulated.

A Repetition Set defines how one or more scenario executions are repeated.

Therefore:

Scenario diversity $\neq$ Repetition

and:

Repetition volume $\neq$ Scenario Coverage.

Deterministic Repetition

A deterministic simulation is expected to produce materially equivalent behavior when materially equivalent execution conditions are reproduced.

Where materially different behavior emerges, SRPM may identify:

Unexpected Execution Divergence.

No Determinism Assumption

A simulation should not be classified as deterministic merely because its design intends determinism.

Actual execution behavior must remain governable.

Stochastic Repetition

A stochastic simulation intentionally permits variability.

SRPM governs: randomization mechanism, random seed, distribution sampling, number of repetitions, relation between repetitions, and whether observed variation is consistent with the intended stochastic structure.

Random Seed

A random seed may determine a stochastic execution path.

Where material, SRPM preserves:

Seed Identity $\rightarrow$ Execution ID

This supports reconstruction and comparison.

Seed Reuse

Seed reuse may be deliberate.

For example, the same stochastic path may be replayed while one parameter

changes.

This can strengthen controlled comparison.

The reuse should remain explicit.

Seed Variation

Different seeds may be used to explore distributional behavior.

The simulation record should preserve which execution used which seed where technically applicable. Seed $\neq$ Reproducibility

SIMULOS® establishes:

Preserving the random seed does not establish reproducibility when other materially relevant execution conditions differ.

Initialization Variation

Repeated simulations may vary initial states.

Examples include: actor placement, resource state, environmental state, memory state, system state, mission phase, or another initialization condition.

Where initialization varies intentionally, it should remain part of the repetition design.

Execution-Order Effects

Some simulation systems may behave differently depending upon execution order because of: shared state, cache behavior, external dependencies, persistent environment state, adaptive models, memory, or resource contention.

SRPM should identify materially relevant execution-order effects.

No Independent-Run Assumption

SIMULOS® establishes:

Separate execution IDs do not prove statistical or operational independence.

Runs may remain dependent through: shared initialization, shared randomization, shared data, shared adaptive state, shared infrastructure, previous-run state, or common external services.

Repetition Independence

Where independence matters, the repetition process should identify whether executions are:

Independent

Conditionally Independent

Dependent

or:

Independence Undetermined.

Shared-State Repetition

Repeated simulations may unintentionally inherit state from earlier runs.

Examples include: memory, cache, learned model state, environment state, persistent agent state, database state, or external service state.

This can make apparently repeated executions materially non-equivalent.

No Hidden State Carryover Principle

A repetition must not be represented as a fresh execution where material state from earlier executions persists invisibly. Repeatability

Repeatability concerns whether materially equivalent executions performed under sufficiently equivalent conditions produce behavior consistent with the simulation's expected execution characteristics.

Repeatability does not necessarily require identical outputs.

The required equivalence depends upon whether the simulation is deterministic, stochastic, adaptive, distributed, or human-in-the-loop.

Repeatability Conditions

Repeatability may depend upon: identical model version, identical environment, identical inputs, identical parameters, identical seed, equivalent hardware, equivalent software dependencies, equivalent timing, and absence of hidden state.

Repeatability Failure

A Repeatability Failure occurs where executions expected to be materially equivalent produce unexplained materially different behavior.

The result should not immediately be treated as simulation invalidity.

It creates a governed question requiring explanation. Reproducibility

Reproducibility concerns whether materially comparable simulation behavior can be recreated under another suitably controlled execution context.

This may involve: another machine, another environment, another organization, another implementation, or a later point in time.

Repeatability ≠ Reproducibility

A simulation may be repeatable on one platform while not reproducible elsewhere.

These remain distinct properties.

Repetition Comparability

Two repeated executions should be treated as comparable only where their materially relevant similarities and differences are known.

Comparability may be:

Fully Comparable

Conditionally Comparable

Partially Comparable

Not Comparable

or:

Comparability Undetermined.

No Same-Scenario Comparability Principle

SIMULOS® establishes:

Sharing the same Scenario ID does not establish that two Simulation Executions are sufficiently comparable.

Material differences may exist in: inputs, parameters, environment version, model version, timing, infrastructure, randomization, or runtime intervention.

Repetition Equivalence

A repeated execution may be considered materially equivalent to another where relevant differences do not materially affect the repetition purpose.

Equivalence should remain tied to the specific comparison question.

Repetition Variance

Repetition Variance describes materially relevant differences across repeated executions.

Variance may be expected.

For example, stochastic simulations may deliberately generate different outcomes.

SRPM asks:

Is the variance expected, explainable, bounded, and traceable to governed

repetition conditions?

Execution Divergence

Execution Divergence occurs where repeated executions move into materially different behavior.

Divergence may arise from: stochasticity, chaotic system behavior, timing, hidden state, model instability, adaptive behavior, environment differences, or uncontrolled execution drift.

Expected vs Unexpected Divergence

SRPM distinguishes:

Expected Divergence

from:

Unexpected Divergence.

Expected divergence follows from the simulation design.

Unexpected divergence requires investigation because materially equivalent execution conditions were expected to behave differently only within known bounds.

Divergence Trigger

A material divergence may create: additional observation requirements, new repetition requirements, new scenario requirements, edge-case discovery, stress-condition requirements, or execution-control review.

Rare-Event Emergence

Repeated simulation may reveal low-frequency behavior absent from earlier runs.

A rare event should not automatically be discarded as:

simulation noise.

SRPM preserves:

Rare Observed Behavior → Execution Identity → Repetition Context

so the condition can be evaluated by downstream governance. Rare Event ≠ Error

SIMULOS® establishes:

Low-frequency behavior must not be classified as execution error solely because it occurred rarely.

Rare-Event Confirmation

A newly observed rare condition may justify additional repetition to determine whether it: recurs, depends upon specific conditions, represents stochastic variation, reflects hidden instability, or reveals a new scenario requirement.

Repetition Convergence

Some simulation campaigns seek evidence that repeated results have stabilized sufficiently for the Simulation Purpose.

A Convergence Signal may indicate that additional repetitions produce diminishing material changes in the relevant result distribution or observed behavior. Convergence $\neq$ Truth

SIMULOS® establishes:

Stable repeated output does not establish that the simulation correctly represents reality.

It establishes only a property of repeated simulation behavior.

Reality correlation and validation remain separate governance questions.

Repetition Saturation

A Repetition Set may reach Repetition Saturation where additional repeated runs provide little additional information for the defined repetition purpose.

This can prevent wasteful run-volume growth.

No Fixed Run-Count Principle

SIMULOS® does not establish that:

10 runs

1,000 runs

or:

1,000,000 runs

are universally sufficient.

Repetition sufficiency depends upon: purpose, stochasticity, uncertainty, variability, rarity, sensitivity, consequence, and the simulation question being examined.

Repetition Sufficiency

Repetition Sufficiency asks:

Have enough controlled repetitions been performed to support the repetition purpose without claiming more than the resulting execution set can demonstrate?

Possible statuses may include:

Sufficient

Conditionally Sufficient

Partially Sufficient

Insufficient

or:

Undetermined.

Repetition Gap

A Repetition Gap exists where additional governed repetition is materially required but has not been completed.

A gap may arise because of: insufficient run count, rare-event uncertainty, unexplained divergence, inadequate stochastic sampling, resource limitations, interrupted campaign, or missing repeatability analysis.

Repetition Failure

A repeated execution may itself fail.

Failed repetitions should remain visible.

A simulation campaign should not preserve:

successful runs only

while removing failures from the repetition basis.

No Survivor-Only Repetition Principle

SIMULOS® establishes:

Repetition governance must preserve materially relevant failed, interrupted, anomalous, and divergent executions rather than allowing only successful repetitions to define the observed simulation behavior.

Repetition Drift

Repetition Drift occurs where repeated executions gradually cease to share the material conditions required for the intended comparison.

Possible causes include: model updates, environment updates, configuration changes, dependency updates, hardware changes, data refresh, adaptive state, or altered execution controls.

Repetition Drift vs Intended Evolution

Not every change is drift.

A repetition campaign may deliberately evolve.

The distinction is:

Governed Change

versus:

Uncontrolled Change affecting repetition comparability.

Version-Bounded Repetition

Repetition Sets should remain bounded to relevant: model versions, environment versions, scenario versions, input versions, and execution-control versions

where changes materially affect comparability.

Cross-Version Repetition

Cross-version comparison may be legitimate where the purpose is explicitly to evaluate change.

For example:

Model Version 1 vs Model Version 2

However, this is not repetition under equivalent conditions.

It is a governed comparative repetition structure.

Baseline Replay

A prior execution may be replayed as a baseline against a changed system version.

This can help isolate the effect of a change where other materially relevant conditions remain controlled.

Repetition Lineage

SRPM preserves lineage such as:

Reference Execution

↓

Repeat 1

↓

Repeat 2

↓

Rare Behavior Detected

↓

Focused Repeat Set

↓

Controlled Reproduction

↓

New Scenario Requirement

This allows repeated simulation to become a source of structured discovery.

Adaptive Repetition

An execution campaign may adapt repetition based on observed results.

For example:

Initial 100 Runs

↓

Rare Divergence Detected

↓

Additional 1,000 Targeted Runs

↓

Behavior Region Identified

The adaptive decision should remain traceable.

No Hidden Adaptive Sampling Principle

Where observed simulation results materially determine which additional repetitions occur, the selection logic should remain visible.

Otherwise the final repetition population may appear unbiased when it was adaptively selected.

Monte Carlo Repetition

Monte Carlo simulation is a major application of SRPM.

SRPM may govern: run identity, seed generation, sampling distributions, sample dependence, repeated configuration, randomization logic, execution failures, coverage of relevant probability regions, and relation between run volume and actual repetition purpose.

Monte Carlo Volume ≠ Evidence Sufficiency

SIMULOS® establishes:

Millions of Monte Carlo runs do not establish methodological sufficiency if the sampling model, scenario basis, execution conditions, or materially relevant low-probability regions remain poorly governed.

Stochastic Tail Repetition

High-consequence behavior may occur in distribution tails.

Random sampling may generate too few such executions for meaningful examination.

SRPM may therefore identify a need for: targeted tail repetition, stratified repetition, controlled seed selection, scenario injection, or another deliberate method.

Scenario design remains governed under SSS.

SRPM governs the repetition structure.

Repeated Edge Cases

An Edge Case identified under SSS may require repetition to determine whether observed behavior is: deterministic, stochastic, unstable, timing-dependent, or conditional upon another execution variable.

Repeated Stress Conditions

Stress scenarios may require repeated execution because system behavior near a failure or degradation boundary may vary.

SRPM can support examination of whether:

Failure always occurs

Failure sometimes occurs

or:

Failure depends upon specific stochastic conditions.

Autonomous-System Repetition

Autonomous systems may produce different behaviors under apparently similar conditions because of: nondeterministic decision processes, adaptive state,

memory, multi-agent interaction, environment timing, or stochastic components.

SRPM preserves the distinction between:

same scenario

and:

same execution path.

AI Agent Repetition

An AI agent may be executed repeatedly on the same task while producing materially different: plans, tool calls, escalation behavior, recovery behavior, or final actions.

SRPM can govern: prompt or task identity, model version, tool state, permissions, memory state, environment state, temperature or sampling configuration, and repetition results.

This helps reveal behavioral dispersion that a single successful run cannot expose.

Space and Mission Simulation

Space simulation programs frequently rely upon large repeated campaigns to examine uncertainty in: trajectory, navigation, component behavior, environment, communication, timing, power, sensor error, and mission recovery.

SRPM asks:

Are these repetitions actually controlled enough to interpret their differences?

For example, if a spacecraft recovery simulation fails in:

7 of 100,000 runs

the relevant questions include:

Which seven?

Which seeds?

Which input combinations?

Did those runs share a dependency state?

Can the behavior be reproduced?

Was the same environment version used?

Did any run contain an execution deviation?

The value lies not in the number:

100,000

but in the governed ability to understand what the repetitions collectively demonstrate.

Industrial Autonomous Systems

A manufacturing robot may pass:

999 of 1,000 repetitions

of one scenario.

The remaining execution may reveal a rare coordination failure.

SRPM prevents that execution from being discarded merely because:

99.9% succeeded.

Instead the rare behavior can be: reconstructed, repeated, isolated, and transferred to Scenario Design if a new material condition is discovered.

Repetition Campaign Diagnostic

SRPM can be used on an existing simulation program to ask:

Why are these runs repeated?

Which variables remain constant?

Which variables intentionally change?

Which variables changed unintentionally?

Are the runs independent?

Are seeds traceable?

Are failed runs preserved?

Are repeated executions actually comparable?

What variance is expected?

What divergence is unexplained?

Has repetition reached saturation?

Are millions of executions producing materially new information?

This can expose substantial inefficiency and weak execution governance without changing the simulation engine itself.

Relationship to SIM

SIM — Simulation Instantiation Module establishes the exact simulation instance.

SRPM depends upon that execution identity to determine whether a later run is

sufficiently comparable to a prior one.

Relationship to SECM

SECM — Simulation Execution Control Module governs the active control conditions applied during each run.

SRPM determines whether those execution controls remained sufficiently equivalent across repetitions.

Relationship to SOM

SOM — Simulation Observation Module captures materially relevant behavior during each execution.

SRPM compares the resulting observations across a Repetition Set.

Relationship to SEDM

SEDM — Simulation Execution Documentation Module preserves the repetition records, lineage, differences, and resulting execution history.

SRPM establishes the repetition governance logic.

SEDM ensures that logic and its execution record remain reconstructable.

Relationship to SSS

SSS governs:

which scenarios and condition spaces should be examined.

SRPM governs:

how an execution of those scenarios is repeated.

Therefore:

Scenario Coverage $\neq$ Repetition Sufficiency.

Both may be required, but they govern different methodological questions.

Relationship to SIDS

SIDS governs the suitability of Simulation Inputs.

SRPM preserves which exact Input Set or input variations applied across repeated executions.

Relationship to SMES

SMES governs the Simulation Model & Environment.

SRPM relies upon model and environment version binding to determine whether

repeated runs remain comparable.

Relationship to SOGS

SOGS governs the outputs produced by repeated executions.

SRPM establishes the repetition basis needed to determine whether outputs represent: one run, one Repetition Set, a distribution, a rare execution, an aggregated result, or another repetition-derived output structure.

Relationship to VALIDOS®

Repeated simulation may later become part of a Validation Evidence Basis.

VALIDOS® determines what that repetition evidence can legitimately support.

SRPM does not establish formal validation.

Relationship to ADIT®

ADIT® may audit whether repetition records, run populations, seed usage, execution failures, and comparability claims are sufficiently supported.

SRPM governs repetition itself.

Governance Boundary

SRPM begins when one or more governed Simulation Executions require deliberate repetition for a defined simulation purpose.

SRPM ends when repeated executions have been sufficiently: identified, related, controlled, differentiated, compared, variance-assessed, divergence-assessed, sufficiency-assessed, and preserved as a governed Repetition Set

for downstream Execution Documentation and Output Governance.

SRPM does not determine: general Scenario Coverage, model validity, output interpretation, formal validation, or operational reliance.

Minimum Implementation Framework

1. Establish the Repetition Requirement

Define why repeated execution is required. 2. Identify the Reference Execution

Bind the repetition process to a sufficiently defined execution basis. 3.
Define Controlled Constants

Identify the materially relevant conditions intended to remain unchanged. 4.
Define Permitted Variation

Identify which conditions may intentionally change across repetitions. 5.
Establish Repetition Identity

Assign distinct Execution IDs and a common Repetition Set relationship. 6. Govern Randomization

Where applicable, preserve: randomization method, seed, distribution, and relevant stochastic configuration.

7. Execute Controlled Repetitions

Perform repeated runs while preserving intended constants and permitted variation. 8. Detect Repetition Drift

Identify material unintended changes affecting comparability. 9. Assess Variance and Divergence

Distinguish expected stochastic variation from unexplained execution divergence. 10. Preserve Failed and Rare Executions

Ensure materially relevant anomalous, failed, interrupted, and low-frequency runs remain part of the repetition record. 11. Assess Repetition Sufficiency

Determine whether additional repetition would materially improve the defined simulation inquiry. 12. Establish the Governed Repetition Set

Preserve the complete repetition basis for Execution Documentation and downstream Output Governance.

Governance Outputs

SRPM may produce: Repetition Requirement, Reference Execution Record, Repetition Set Identifier, Simulation Repetition Set, Controlled Constant Record, Permitted Variation Record, Repetition Execution Record, Repetition Identity Map, Randomization Record, Random Seed Record, Seed-Reuse Record, Seed-Variance Record, Initialization Variation Record, Execution-Order Effect Record, Repetition Independence Assessment, Shared-State Record, Repeatability Assessment, Repeatability Failure Record, Reproducibility Condition Record, Repetition Comparability Assessment, Repetition Equivalence Record, Repetition Variance Record, Execution Divergence Record, Rare-Event Observation Record, Rare-Event Confirmation Requirement, Convergence Signal, Repetition Saturation Assessment, Repetition Sufficiency Assessment, Repetition Gap, Repetition Failure Record, Repetition Drift Record, Cross-Version Repetition Record, Adaptive Repetition Record, Repetition Lineage Map, and Governed Simulation Repetition Set.

Use Case 1 — Spacecraft Autonomous Recovery Monte Carlo Campaign

Scenario

A spacecraft autonomous-recovery simulation executes 250,000 repeated runs under uncertainty in: navigation error, communication delay, power state, sensor noise, and recovery timing.

A small number of executions fail to recover correctly. Application

SRPM preserves: Repetition Set identity, individual Execution IDs, seeds, input and parameter combinations, environment version, controlled constants, permitted stochastic variation, and failed-run lineage.

The failed executions are compared with successful repetitions.

A common pattern is discovered:

high navigation uncertainty + delayed communication + low remaining power at recovery initiation. Result

The organization can move beyond:

249,982 successful / 18 unsuccessful runs

to determine:

which governed conditions are associated with the rare failure behavior and whether that behavior can be deliberately reproduced.

The 18 executions may then create new Scenario or Stress-Condition requirements rather than disappearing inside aggregate statistics. Use Case 2 — Autonomous Robot Repetition Divergence

Scenario

An industrial autonomous robot is simulated repeatedly under what is intended to be the same degraded-sensor scenario.

Most runs behave consistently.

A small subset produces different fallback behavior. Application

SRPM compares: model version, environment version, input basis, parameterization, randomization, memory state, and prior execution state.

The analysis discovers that persistent agent state was not reset between some repetitions. Result

The behavior is not incorrectly classified as inherent stochastic variation.

It is identified as:

Hidden State Carryover → Repetition Comparability Failure.

The simulation program can correct the repetition process and rerun the affected executions under controlled conditions.

Architectural Position

Simulation Repetition Module (SRPM) is the fourth internal governance space of the Simulation Execution Standard (SES).

The complete SES progression is:

Simulation Instantiation

↓

Simulation Execution Control

↓

Simulation Observation

↓

Simulation Repetition

↓

Simulation Execution Documentation

SIM establishes what exact simulation instance enters execution.

SECM governs how that instance is controlled while running.

SOM governs what materially relevant behavior occurs during execution.

SRPM governs how executions are deliberately repeated and meaningfully compared.

SEDM then preserves the complete execution and repetition history as a reconstructable governed record.

Foundational Principle

Repeated simulation creates methodological value only when the conditions that remained constant, the conditions that changed, the dependencies between runs, and the reasons for observed variance remain sufficiently governed to distinguish meaningful repetition from mere execution volume.

Canonical Closing Statement

Simulation Repetition Module establishes the repetition governance layer of the Simulation Execution Standard by defining how governed Simulation Executions are repeated, related, randomized, compared, and assessed for repeatability, reproducibility conditions, variance, divergence, rare behavior, convergence, saturation, and sufficiency. Through controlled constants, permitted variation, run identity, seed governance, independence assessment, hidden-state detection, repetition drift control, preservation of failed and anomalous runs, adaptive repetition, and complete repetition lineage, SRPM prevents high execution counts from being mistaken for methodological strength and ensures that repeated simulations remain interpretable as a governed body of execution evidence rather than an

undifferentiated collection of runs.

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>